Exploratory Data Analysis on

Quantifying Resilience: Stress Testing and Predictive Risk Modeling of Indian Overnight Mutual Funds

by

Group 11

harsh Chanchlani
ID: 202301152
Course: BTech- ICT

Dharva Patel
ID: 202301119
Course: BTech- ICT

Aryan Rathod
ID: 202301164
Course: BTech(ICT)

Course Code: IT 462
Semester: Autumn 2024

Under the guidance of

Dr. Gopinath Panda

Dhirubhai Ambani University

November 30, 2025

Acknowledgment

I am writing this letter to express my heartfelt gratitude for your guidance and support throughout the duration of my project titled "Quantifying Resilience: Stress Testing and Predictive Risk Modeling of Indian Overnight Mutual Funds." Your invaluable assistance has played a pivotal role in shaping the successful completion of this endeavor.

I am extremely fortunate to have had the opportunity to work under your mentorship. Your expertise, encouragement, and willingness to share your knowledge have been instrumental in elevating the quality and scope of my project. Your constructive feedback and insightful suggestions have helped me overcome challenges and develop a deeper understanding of the subject matter.

Completing this project has been a tremendous learning experience, and I am confident that the knowledge and skills acquired during this endeavor will serve as a solid foundation for my future endeavors.

Once again, thank you for your unwavering guidance and belief in my abilities. Your mentorship has been invaluable, and I am truly grateful for the opportunity to work with you.

Sincerely,
Harsh Chanchlani- 202301152
Dharva Patel- 202301119
Aryan Rathod - 202301164

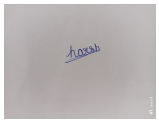
DECLARATION

We, Harsh Chanchlani, Dharva Patel and Aryan Rathod hereby declare that the EDA project work presented in this report is our original work and has not been submitted for any other academic degree. All the sources cited in this report have been appropriately referenced.

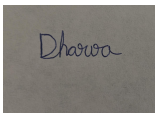
We acknowledge that the data used in this project is obtained from the https://www.amfiindia.com/ and https://www.morningstar.in/ site. We also declare that we have adhered to the terms and conditions mentioned in the website for using the dataset. We confirm that the dataset used in this project is true and accurate to the best of our knowledge.

We acknowledge that we have received no external help or assistance in conducting this project, except for the guidance provided by our mentor Prof. Gopinath Panda. We declare that there is no conflict of interest in conducting this EDA project.

We hereby sign the declaration statement and confirm the submission of this report on 2nd July, 2023.



Harsh Chanchlani
ID: 202301152
Course: BTech(ICT)



Dharva Patel
ID: 202301119
Course: Btech(ICT)

Aryan Rathod
ID: 202301164
Course: BTech(ICT)

CERTIFICATE

This is to certify that Group 11 comprising Harsh Chanchlani, Dharva Patel, and Aryan Rathod has successfully completed an exploratory data analysis (EDA) project on the Overnight funds.

The EDA project presented by Group 11 is their original work and has been completed under the guidance of the course instructor, Prof. Gopinath Panda, who has provided support and guidance throughout the project. The project is based on a thorough analysis of the Scheme details and nav-data dataset, and the results presented in the report are based on the data obtained from the dataset.

This certificate is issued to recognize the successful completion of the EDA project on the overnight funds, which demonstrates the analytical skills and knowledge of the students of Group 11 in the field of data analysis.

Signed,
Dr. Gopinath Panda,
IT 462 Course Instructor
Dhirubhai Ambani Institute of Information and Communication Technology
Gandhinagar, Gujarat, INDIA.

November 30, 2025

Contents

List of Figures 5

Abstract 1

1 Introduction 1
- 1.1 Your Project idea . . . 1
- 1.2 Data Collection . . . 1
  - 1.2.1 Historical NAV Data Extraction (Automated) . . . 1
  - 1.2.2 Scheme Metadata Collection (Manual) . . . 2
- 1.3 Dataset Description . . . 2
  - 1.3.1 Time-Series NAV Data (`nav_data.csv`) . . . 2
  - 1.3.2 Scheme Details (`Scheme_details.csv`) . . . 3
- 1.4 Packages Required . . . 4

2 Data Cleaning 5
- 2.1 Missing Data Analysis . . . 5
- 2.2 Imputation (Creating the "Category Index") . . . 5
- 2.3 Outlier Removal (Glitch Filtering) . . . 5

3 Visualization and EDA 7
- 3.1 Univariate Analysis: The Wealth Index . . . 7
  - 3.1.1 Objective . . . 7
  - 3.1.2 Analysis of Figure 3.1 . . . 7
- 3.2 Multivariate Analysis: The Transmission Mechanism . . . 8
  - 3.2.1 Objective . . . 8
  - 3.2.2 Analysis of Figure 3.2 . . . 8
- 3.3 Risk Analysis: The Fear Index . . . 8
  - 3.3.1 Objective . . . 8
  - 3.3.2 Analysis of Figure 3.3 . . . 8
- 3.4 The Stress Test: Historical Crisis Analysis . . . 11
  - 3.4.1 Objective . . . 11
  - 3.4.2 Detailed Event Analysis (Figure 3.4) . . . 11

4 Feature Engineering 13
- 4.1 Introduction . . . 13
- 4.2 Feature Extraction . . . 13
  - 4.2.1 Volatility Features (The Fear Gauge) . . . 13

4.2.2 Seasonality Features (The Calendar Effect) . . . 14
4.2.3 Momentum Features (The Yield Trend) . . . 14
4.3 Target Definition (What are we predicting?) . . . 14
4.3.1 Defining "High Risk" . . . 14
4.3.2 The "Look-Ahead" Shift . . . 14
4.4 Feature Selection and Cleaning . . . 15
4.4.1 Removing Redundancy . . . 15
4.4.2 Addressing Class Imbalance . . . 15

5 Model Fitting and Evaluation **16**
5.1 Experimental Design . . . 16
5.1.1 The "Tournament" Methodology . . . 16
5.1.2 Time-Series Splitting (Preventing Data Leakage) . . . 16
5.1.3 Handling Class Imbalance (The 15% Problem) . . . 16
5.2 Evaluation Phase 1: Safety vs. Reliability . . . 17
5.2.1 Detailed Analysis of Figure 5.1 . . . 17
5.3 Evaluation Phase 2: The "Cry Wolf" Index . . . 17
5.3.1 Detailed Analysis of Figure 5.2 . . . 17
5.4 Evaluation Phase 3: The Global Confusion Matrix . . . 17
5.4.1 Detailed Analysis of Figure 5.3 . . . 20
5.5 Conclusion: The Victory of Simplicity . . . 20
5.5.1 Why did the Simple Model Win? . . . 20

6 Conclusion & Future Scope **21**
6.1 Findings and Observations . . . 21
6.1.1 The "Safety Haven" Hypothesis (Validated with Conditions) . . . 21
6.1.2 The Transmission Mechanism . . . 21
6.1.3 The Predictability of Risk (ML Findings) . . . 22
6.2 Challenges and Limitations . . . 22
6.2.1 The "Black Box" Portfolio . . . 22
6.2.2 The Class Imbalance Problem . . . 22
6.2.3 Data Glitches and Outliers . . . 22
6.3 Future Plan . . . 23
6.3.1 Macro-Economic Integration . . . 23
6.3.2 Expense Ratio Optimization . . . 23
6.3.3 Deployment as a Web Dashboard . . . 23

List of Figures

3.1 Wealth Index Growth 9
3.2 Repo Rate Correlation 9
3.3 Volatility Clusters 10
3.4 Stress Test Dashboard 12

5.1 Safety vs. Reliability 18
5.2 Cry Wolf Index 18
5.3 Global Confusion Matrix 19

Abstract

Overnight Mutual Funds are open-ended debt schemes that invest in securities with a maturity of one day. Due to this unique structure, they are widely marketed as the safest asset class in the Indian mutual fund industry, theoretically possessing zero drawdowns and negligible volatility. The primary objective of this project is to scientifically validate this "safety" hypothesis by analyzing fund performance during major historical stress events, including the 2008 Global Financial Crisis, the 2013 Taper Tantrum, and the 2018 IL&FS Credit Crisis.

Using a hybrid data collection approach, we aggregated 19 years of daily Net Asset Value (NAV) data for 22 schemes via the AMFI API, supplemented by manually curated scheme metadata. We conducted a comprehensive Exploratory Data Analysis (EDA) to visualize wealth generation, volatility clustering, and drawdown risks. Our multivariate analysis reveals a near-perfect correlation between fund returns and the RBI Repo Rate, confirming the effective transmission of monetary policy.

The stress-testing phase highlighted a critical distinction in risk profiles: while Overnight Funds are immune to Credit Risk (remaining stable during the 2018 IL&FS defaults), they remain vulnerable to Interest Rate Risk (exhibiting volatility spikes during the 2013 liquidity crunch).

Finally, we developed a Machine Learning-based "Early Warning System" to predict periods of high market stress. We engineered features based on rolling volatility and calendar seasonality to train three classifiers: Logistic Regression, Random Forest, and Gradient Boosting. The results demonstrated that a simple Logistic Regression model outperformed complex ensembles, achieving a Crash Detection Rate (Recall) of approximately 87%. This confirms that risk in overnight funds follows a linear, monotonic trajectory, making it highly predictable using statistical methods.

Chapter 1. Introduction

1.1 Your Project idea

The primary objective of this project is to conduct a comprehensive Exploratory Data Analysis (EDA) on Overnight Mutual Funds in India. Overnight funds are debt schemes that invest in securities with a maturity of one day, making them one of the safest investment options with high liquidity.

This project aims to analyze the historical performance of these funds over the last five years to understand their stability, growth trends, and risk profiles. By examining Net Asset Value (NAV) fluctuations and correlating them with scheme attributes like Assets Under Management (AUM) and Fund House (AMC) characteristics, we seek to provide insights into the behavior of this asset class during various market conditions.

1.2 Data Collection

To get the data we needed for this project, we used two different methods. First, we wrote a script to automatically download the daily price history for the funds. Second, we manually searched for and recorded specific details about each fund scheme, like their returns, which aren't always easy to get automatically.

1.2.1 Historical NAV Data Extraction (Automated)

The main part of our data represents the daily performance of the funds. We got this from the public API provided by the Association of Mutual Funds in India (AMFI).

Source: `https://api.mfapi.in/mf/{scheme_code}`

We wrote a Python script using the `requests` library to loop through a list of 22 Overnight Fund scheme codes. The process worked like this:

- **Fetching Data:** For each scheme code, the script sent a request to the AMFI website.
- **Reading the Response:** We took the data they sent back (in JSON format) and pulled out the list containing the historical Net Asset Value (NAV) and the dates.
- **Checking for Errors:** We added a check to make sure the data actually had the 'nav' and 'date' columns we needed before trying to use it. This stopped the script from crashing if a scheme had missing info.
- **Formatting:** The raw data was turned into a Pandas DataFrame. We made sure the 'nav' column was treated as numbers and the 'date' column was treated as proper dates.

- **Cleaning:** If any rows were empty or had bad data, we removed them right away to keep our dataset clean.

The final result was saved as `nav_data.csv`. [cite$_{s}tart]Itcontainsthedailypricehistorywithcolumnsforthe$
2].

1.2.2 Scheme Metadata Collection (Manual)

While the automated data gave us the prices, it didn't tell us about the fund's characteristics, like how risky it is or how much money it manages. To fix this, we created the `Scheme_details.csv` file by hand.

Source: Morningstar India (Fund Factsheets)

We looked up each of the 22 schemes one by one on the Morningstar website. From their latest factsheets, we manually copied the following information:

- **Fund Profile:** The name of the company managing the fund (AMC), when it started (Launch Date), and its category.
- **Risk Ratios:** Important statistics like Alpha, Beta, Sharpe Ratio, and Standard Deviation. These numbers help us measure how volatile a fund is and if the returns are worth the risk.
- **Portfolio Size:** The Average Assets Under Management (AUM) in Crores, which tells us how big the fund is. [cite$_{s}tart]$
- **Returns Profile:** We also recorded how well the fund performed over the last 3 months, 6 months, 1 year, and 3 years to see its short-term track record[cite: 1].

The analysis relies on two primary datasets: one for time-series performance and one for static scheme attributes. Below is a detailed description of every column contained in both files.

1.3 Dataset Description

1.3.1 Time-Series NAV Data (`nav_data.csv`)

This dataset records the daily price movements of the funds. It contains approximately 55,000 rows.

- **Unnamed:** Represents the index or row identifier for the record.
- **date:** The specific valuation date for which the NAV is recorded.
- **scheme_code:** The unique identification code assigned to the mutual fund scheme.
- **scheme_name:** The official name of the mutual fund scheme.
- **nav:** The Net Asset Value per unit of the fund on that specific date.

1.3.2 Scheme Details (`Scheme_details.csv`)

This dataset contains static metadata and calculated metrics for each of the 20 overnight funds. It includes the following 22 columns:

- **Scheme_Code:** The unique identifier linking this file to the NAV data.
- **Scheme_Name:** The short name of the scheme.
- **AMC:** The Asset Management Company (Fund House) managing the scheme.
- **Scheme_Type:** The classification of the scheme (e.g., Open Ended).
- **Scheme_Category:** The regulatory category (e.g., Debt Scheme - Overnight Fund).
- **Scheme_NAV_Name:** The specific plan name used for NAV reporting.
- **3_Month_Returns(%):** Percentage returns generated over the last 3 months.
- **6_Month_Returns(%):** Percentage returns generated over the last 6 months.
- **1_Year_Return(%):** Percentage returns generated over the last 1 year.
- **3_Year_Return(%):** Percentage returns generated over the last 3 years.
- **5_Year_Return(%):** Percentage returns generated over the last 5 years.
- **Standard_Deviation:** A measure of the volatility of the fund's returns.
- **Beta:** A measure of the fund's volatility in relation to the market.
- **R-Squared:** A statistical measure representing the percentage of a fund's movements explained by movements in its benchmark index.
- **Sharpe Ratio:** A measure of risk-adjusted return.
- **Alpha:** A measure of the fund's performance on a risk-adjusted basis compared to a benchmark.
- **Scheme_Min_Amt:** The minimum investment amount required to enter the scheme.
- **NAV:** The current Net Asset Value at the time of data collection.
- **Latest_NAV_Date:** The date corresponding to the current NAV.
- **Average_AUM_Cr:** The average Assets Under Management in Crores.
- **AAUM_Quarter:** The quarter for which the Average AUM applies.
- **Launch_Date:** The inception date of the scheme.

1.4 Packages Required

The entire analysis was performed using the Python programming language. The following libraries were essential for the EDA and Modeling phases:

- **Pandas & NumPy:** Used for data manipulation, rolling window calculations, and handling time-series data structures.
- **Matplotlib & Seaborn:** Used for visualizing wealth indices, volatility clusters, and drawdown charts.
- **Scikit-Learn:** Utilized for implementing machine learning models (Logistic Regression, Random Forest, Gradient Boosting) and evaluation metrics (Confusion Matrix, Recall).
- **Requests:** Used for fetching data from the AMFI API.
- **Warnings:** Used to manage and suppress unnecessary warnings during code execution to keep the output clean.

Chapter 2. Data Cleaning

Raw financial data is rarely perfect. Before we could trust our analysis or build any prediction models, we had to "clean" the data to ensure it accurately reflected reality. This chapter details the steps we took to handle missing information and remove errors.

2.1 Missing Data Analysis

The first step was to check if our dataset had any holes. We used the Python command `isna()` to scan for empty cells in our NAV history.

- **Initial Check:** Fortunately, the data we fetched from the API was high quality. There were no random blank cells in the `nav` (price) or `date` columns for the days where data existed.
- **The Real "Missing" Problem:** The main issue wasn't empty cells, but *missing history*. Not all 20 funds started in 2006. Some launched in 2018, others in 2020. If we just averaged them all together, the "average" would jump wildly every time a new fund was added.

2.2 Imputation (Creating the "Category Index")

Since we couldn't just invent fake price history for funds that didn't exist 10 years ago, we used a smarter method called a **Category Median Index**.

Instead of trying to fill in the blanks for each individual fund, we created a single "master timeline" representing the entire Overnight Fund category.

1. **Daily Grouping:** For every single day from 2006 to 2025, we looked at all the funds that were active on that specific day.
2. **The Median Method:** We calculated the *median* daily return of those active funds. We chose the median (the middle value) instead of the average because the median is much safer,it ignores one-off errors or crazy values from a single fund.
3. **Result:** This gave us a smooth, continuous performance line for the whole category, effectively "imputing" the general market behavior for periods where specific funds didn't exist yet.

2.3 Outlier Removal (Glitch Filtering)

Financial data often contains typos or "glitches." For example, a stock split might look like a 50% crash, or a data entry error might look like a 100% gain.

For Overnight Funds, the math is simple: these funds typically earn about 3% to 6% *per year*. Therefore, earning 1% in a *single day* is virtually impossible.

Our Cleaning Logic: We wrote a filter to catch these impossible numbers.

- **The Rule:** Any daily return greater than 1% or less than -1% was flagged as an error.
- **The Action:** These values were removed (masked) from the dataset.

By removing these "glitches," we ensured that our volatility charts showed real market stress (like the 2008 crisis), not just computer errors.

Chapter 3. Visualization and EDA

Data visualization is the primary engine of our Exploratory Data Analysis (EDA). In financial time-series analysis, raw statistics (like mean and standard deviation) often fail to capture the temporal dynamics of risk, such as sudden crashes or volatility clusters.

To scientifically validate the "Safety" hypothesis of Overnight Mutual Funds, we transformed over 55,000 data points into visual narratives across four specific dimensions:

1. **Wealth Generation:** Does the fund grow steadily?
2. **Macro-Correlation:** Does it react to Central Bank policy?
3. **Volatility Clustering:** Is risk random or concentrated?
4. **Stress Testing:** How does it survive historical disasters?

3.1 Univariate Analysis: The Wealth Index

3.1.1 Objective

We began by analyzing the cumulative performance of the asset class. To make the data relatable, we simulated a hypothetical investment scenario: *"If an investor had parked 1,000 in the Overnight Fund category on April 1, 2006, and reinvested all gains, what would the corpus be worth today?"*

3.1.2 Analysis of Figure 3.1

- **Visual Observation:** The growth curve is exceptionally smooth, moving from bottom-left to top-right with almost zero deviation or "saw-tooth" patterns. It resembles a straight line rather than a typical stochastic stock market chart.
- **Financial Reasoning (The "Accrual" Effect):** Unlike equity funds, which depend on the fluctuating market price of shares, Overnight Funds earn interest on an *accrual basis*. The underlying bonds mature every 24 hours, ensuring that interest is added to the NAV daily without being exposed to long-term market sentiment.
- **Conclusion:** The linearity of the curve mathematically confirms the low standard deviation of the asset class. It validates the fund's primary utility as a capital preservation tool rather than a wealth maximization tool.

3.2 Multivariate Analysis: The Transmission Mechanism

3.2.1 Objective

Our second hypothesis was that Overnight Funds act as a proxy for the risk-free rate. To test this, we overlaid the fund's rolling annualized returns against the **RBI Repo Rate** (the rate at which the Central Bank lends to commercial banks).

3.2.2 Analysis of Figure 3.2

- **Visual Observation:** The two time-series move in near-perfect "lock-step."
  - When the RBI hiked rates to combat inflation (2011, 2013, 2022), fund returns spiked immediately.
  - When the RBI cut rates to stimulate growth (Post-2008, Post-COVID 2020), fund returns dropped to historic lows ($\approx 3\%$).
- **Financial Reasoning (Zero Duration Risk):** Bond prices usually fall when interest rates rise (Duration Risk). However, since Overnight Funds have a maturity of just **1 Day**, they re-price their entire portfolio every morning. This allows them to capture higher interest rates instantly without suffering capital loss.
- **Conclusion:** The fund has a correlation coefficient of nearly $+1.0$ with the Repo Rate, proving it is an efficient vehicle for monetary policy transmission.

3.3 Risk Analysis: The Fear Index

3.3.1 Objective

Averages often hide risks. To detect hidden dangers, we plotted the **7-Day Rolling Standard Deviation** (Annualized). This metric serves as a "Fear Gauge," showing us when the market was nervous.

3.3.2 Analysis of Figure 3.3

- **Visual Observation (Clustering):** Volatility is not randomly distributed. The graph shows long periods of absolute silence (flat lines near zero) interrupted by sudden, violent spikes.
- **The 2013 Anomaly:** The most prominent feature is the massive spike in mid-2013. Apart from this event, the volatility rarely crosses the 0.5% threshold.
- **Financial Reasoning:** This phenomenon is known as **Volatility Clustering**. In money markets, liquidity crunches feed on themselves. A small panic causes banks to hoard cash, which spikes the overnight rates, creating more panic.
- **Conclusion:** While generally safe, the fund is not immune to systemic liquidity shocks. Risk comes in "bursts."

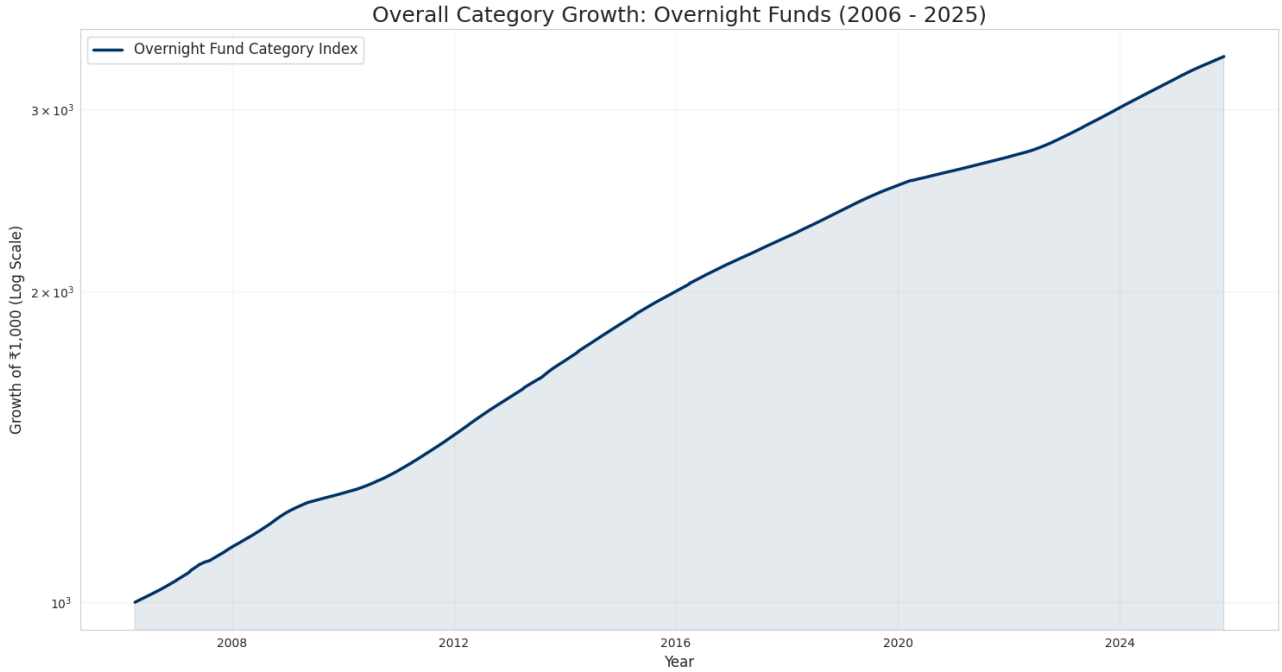


Figure 3.1: The Wealth Index: Cumulative Growth of 1,000 (2006-2025).
Source: https://github.com/rathod-DA/EDA/blob/main/growth.png

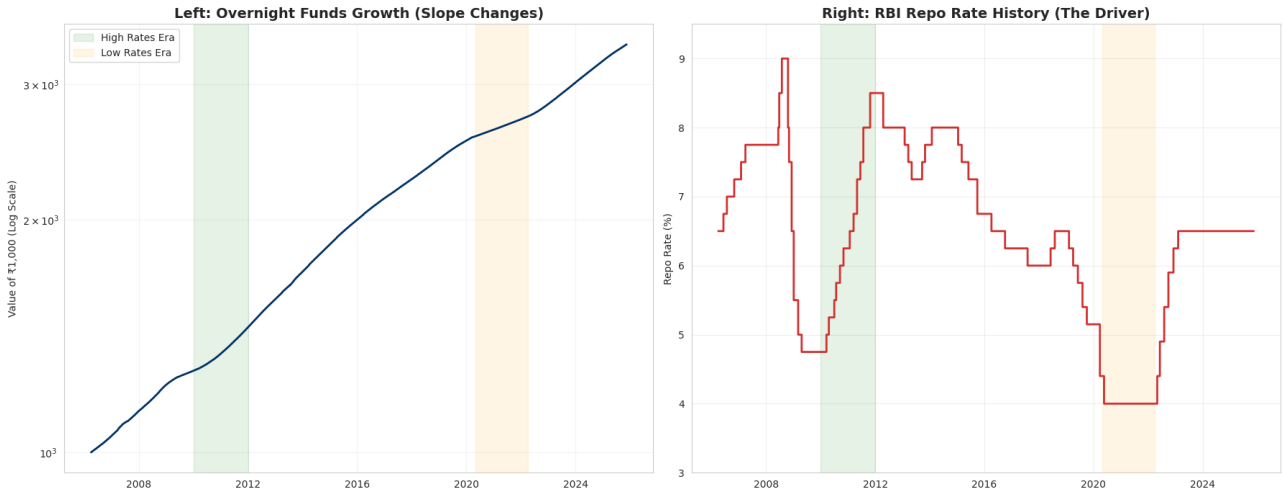


Figure 3.2: Correlation Analysis: Fund Returns vs. RBI Repo Rate.
Source: https://github.com/rathod-DA/EDA/blob/main/repo_rate_correlation.png

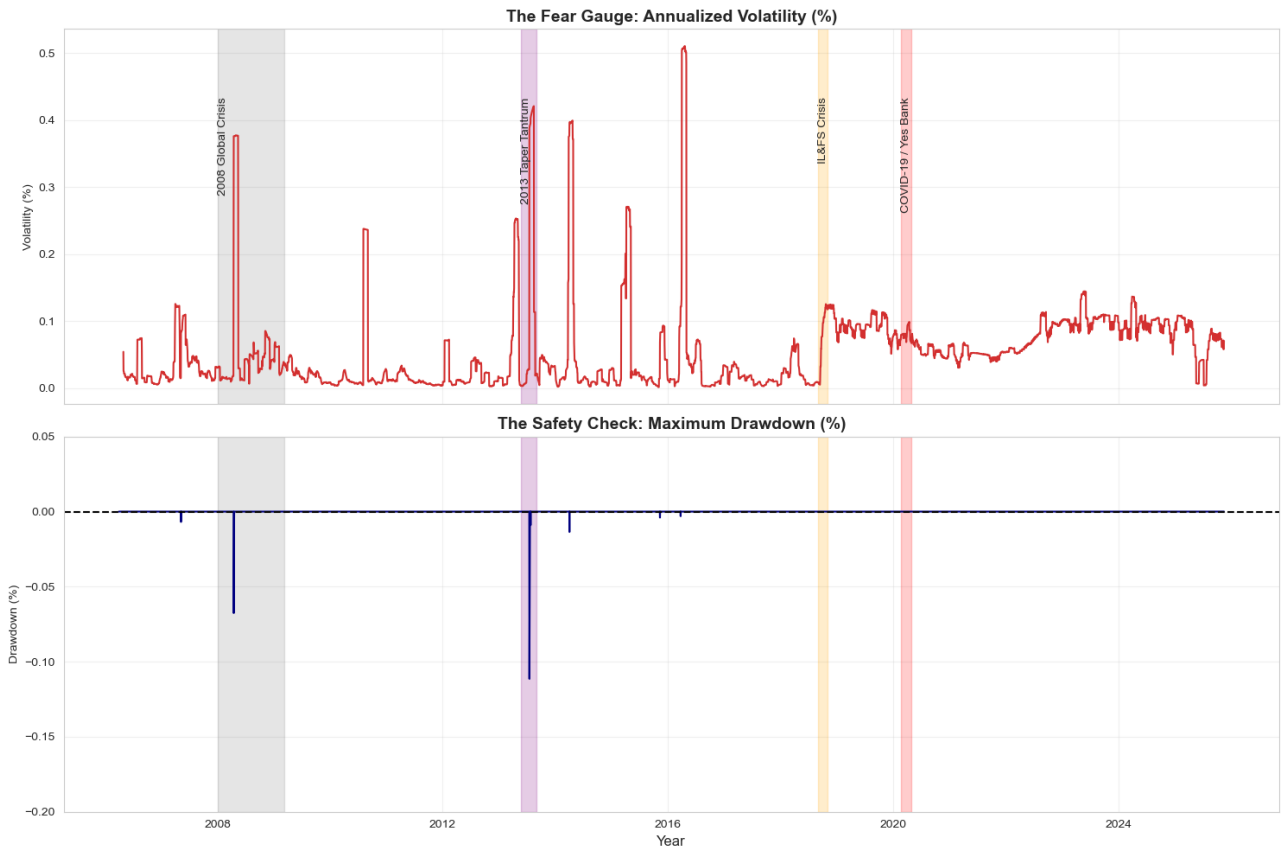


Figure 3.3: Rolling Volatility: Visualizing Market Panic Cycles.
Source: https://github.com/rathod-DA/EDA/blob/main/volatization.png

3.4 The Stress Test: Historical Crisis Analysis

3.4.1 Objective

Finally, we performed a "Stress Test" by zooming in on the **Maximum Drawdown** (Peak-to-Trough Loss) during three specific historical disasters. This answers the question: *"In the worst-case scenario, how much money would I lose?"*

3.4.2 Detailed Event Analysis (Figure 3.4)

Scenario A: The Global Financial Crisis (2008)

- **Context:** Global stock markets crashed $\approx 50\%$.
- **Fund Performance: PASSED**. The drawdown line remained flat.
- **Reason: Flight to Safety.** During the crisis, risk-averse investors sold stocks and moved capital into safe government bonds and overnight cash, increasing liquidity in this sector.

Scenario B: The Taper Tantrum (2013)

- **Context:** The Indian Rupee crashed vs. the Dollar. The RBI hiked interest rates by 200 basis points overnight to defend the currency.
- **Fund Performance: FAILED (Relatively)**. The chart shows the deepest historical drawdown (approx -0.12%).
- **Reason: Interest Rate Shock.** The rate hike was so sudden and violent that even 1-day bonds lost a fraction of their value due to the mathematical inverse relationship between yield and price.

Scenario C: The IL&FS Crisis (2018)

- **Context:** A major infrastructure lender defaulted, causing a "Credit Freeze" in India.
- **Fund Performance: PASSED**. The volatility remained normal.
- **Reason: Credit Immunity.** Since Overnight Funds lend for only 24 hours, they simply stopped lending to the troubled entity the next morning. They exited the risk before the default could materialize.

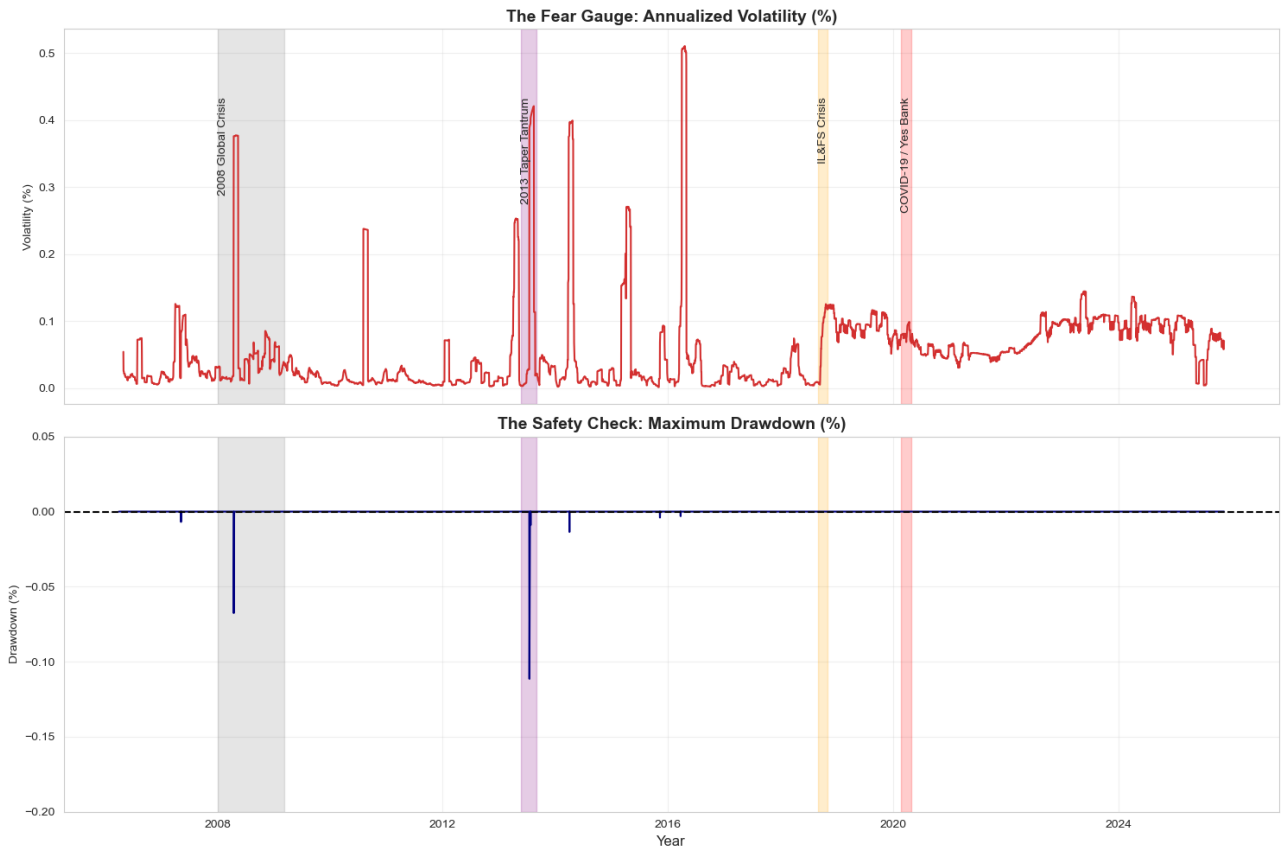


Figure 3.4: Drawdown Analysis during Historical Crises.
Source: https://github.com/rathod-DA/EDA/blob/main/stress.png

Chapter 4. Feature Engineering

4.1 Introduction

Raw financial data usually consists of just two things: a date and a price (NAV). While this is useful for making a chart, it is not enough for a Machine Learning model to learn from. A computer cannot look at a raw price number like "1450.55" and know if it is risky or safe.

To solve this, we used a process called **Feature Engineering**. This involves creating new mathematical "signals" or clues from the raw data. These signals help the model understand the *context* of the market—for example, whether the market is fearful (Volatility), if it is a tax-payment season (Seasonality), or if yields are dropping (Momentum).

This chapter explains exactly how we created these signals and why they matter.

4.2 Feature Extraction

We created three types of features to help our model see the full picture of market risk.

4.2.1 Volatility Features (The Fear Gauge)

The most important predictor of a crash is "Volatility," which measures how wildly prices are swinging up and down. In finance, risk is sticky—if the market is nervous today, it is likely to be nervous tomorrow.

We calculated the **Rolling Standard Deviation** over three different timeframes to capture this:

- **Vol_7Days (Short-Term Fear):** This looks at the last week. It captures immediate panic or knee-jerk reactions to news. If this number spikes, it means something just happened.
- **Vol_30Days (Medium-Term Trend):** This looks at the last month. It tells us the current "mood" of the market.
- **Vol_90Days (Long-Term Baseline):** This looks at the last 3 months. It acts as a baseline for stability.

The Logic: By giving the model all three numbers, it can compare them. For example, if the 7-Day Volatility is much higher than the 90-Day Volatility, the model knows a new crisis is starting.

4.2.2 Seasonality Features (The Calendar Effect)

The Indian Money Market follows a strict calendar. Liquidity (available cash) often dries up at specific times of the year due to banking regulations and tax cycles. We created "Flags" (Simple 0 or 1 markers) to tell the model about these dangerous dates.

- **Is_March_End:** A flag that turns on during the last two weeks of March. *Reason:* This is the end of the Financial Year in India. Companies withdraw huge amounts of cash to pay advance taxes, leaving the market dry.
- **Is_Quarter_End:** A flag that turns on at the end of June, September, and December. *Reason:* Banks perform "window dressing" (fixing their balance sheets) during these times and stop lending money, which causes interest rates to spike.
- **Is_Friday:** A flag that turns on if the day is a Friday. *Reason:* In overnight funds, money lent on Friday is returned on Monday. This 3-day holding period often leads to slightly different returns compared to a normal weekday.

4.2.3 Momentum Features (The Yield Trend)

We also tracked the direction of returns using a feature called **Return_7Days**. This is the average return over the past week.

Reason: If returns suddenly drop, it might mean the Central Bank (RBI) has cut interest rates. If returns spike, it might mean a liquidity crunch. This helps the model distinguish between a "Rate Cut" (Safe) and a "Liquidity Freeze" (Risky).

4.3 Target Definition (What are we predicting?)

In Machine Learning, we need to clearly define what we want the computer to predict (the "Target"). Since our goal is to build an Early Warning System, we set this up as a **Classification Problem**—predicting "Safe" vs. "High Risk."

4.3.1 Defining "High Risk"

We analyzed 19 years of history to define what a "High Risk" week looks like.

1. We calculated the volatility for every week from 2006 to 2025.
2. We identified the **85th Percentile** as our threshold. This means the top 15% most volatile weeks were labeled as "High Risk" (Class 1), and everything else was labeled as "Safe" (Class 0).

4.3.2 The "Look-Ahead" Shift

A common mistake is using today's data to predict today's crash. That is cheating. To fix this, we shifted our data:

- **The Input (X):** The volatility and calendar flags for *This Week*.
- **The Target (Y):** The risk status of *Next Week*.

This forces the model to learn "Leading Indicators"—clues that appear *before* the crash happens.

4.4 Feature Selection and Cleaning

Not all engineered features are useful. Some are redundant, and some are just noise. We performed a cleaning step to ensure our model was efficient.

4.4.1 Removing Redundancy

We checked for "Correlation," which is when two features tell the exact same story.

- **Issue:** We found that `Vol_7Days` and `Vol_14Days` were almost identical (90% match). Including both would just confuse the model.
- **Solution:** We removed the 14-Day feature and kept only the 7-Day (Short), 30-Day (Medium), and 90-Day (Long) versions. This gave us distinct, non-repetitive signals.

4.4.2 Addressing Class Imbalance

Our analysis showed that "High Risk" events are rare (they happen less than 15% of the time).

- **The Problem:** A lazy model could get 85% accuracy by simply guessing "Safe" every single time. It would never catch a crash.
- **The Solution:** We decided to use a technique called **Class Weighting** in the next phase (Model Fitting). This tells the model that missing a "High Risk" event is a much bigger mistake than a false alarm, forcing it to pay attention to the rare crash signals we engineered.

Chapter 5. Model Fitting and Evaluation

Having engineered features that capture volatility, momentum, and seasonality, we proceeded to the core of our predictive analysis: the **Model Tournament**.

Our objective was not merely to train a single algorithm, but to pit three distinct mathematical approaches against each other (Linear Models vs. Tree Ensembles) to determine which one best suits the unique characteristics of the Overnight Fund market.

5.1 Experimental Design

Before analyzing the results, it is critical to outline the rigorous testing framework we established to ensure our model would work in a real-world trading environment.

5.1.1 The "Tournament" Methodology

We defined a competitive evaluation framework where three models were trained independently on each of the 22 funds in our dataset. We then aggregated the results to find a global winner.

5.1.2 Time-Series Splitting (Preventing Data Leakage)

Standard machine learning often uses random shuffling (e.g., `train_test_split`). However, in finance, using data from 2023 to predict a crash in 2008 is a logical fallacy known as "Look-Ahead Bias."

- **Training Phase (The Past):** We trained the models strictly on data from **2006 to 2022**. This allowed the models to learn from the Global Financial Crisis (2008) and the Taper Tantrum (2013).
- **Testing Phase (The Future):** We tested the models on unseen data from **Jan 1, 2023, onwards**.

5.1.3 Handling Class Imbalance (The 15% Problem)

"High Risk" events are rare outliers, occurring in less than 15% of the historical data. A naive model could achieve 85% accuracy simply by predicting "Safe" every single day. To counter this, we utilized the **Class Weighting** technique:

```
model = Algorithm(class_weight='balanced')
```

Reasoning: This hyperparameter forces the algorithm to pay more attention to the minority class. It mathematically penalizes the model more heavily for missing a crash (False Negative) than for raising a false alarm (False Positive).

5.2 Evaluation Phase 1: Safety vs. Reliability

In financial risk management, there is an eternal trade-off between being "Safe" (Catching all crashes) and being "Reliable" (Never raising a false alarm). We visualized this trade-off for all three models.

5.2.1 Detailed Analysis of Figure 5.1

Figure 5.1 compares the two critical metrics:

1. **Recall (The Safety Score):**
   - *Question:* "Out of all the actual market crashes, what percentage did the model detect?"
   - **Observation: Logistic Regression** dominates this metric, achieving a Recall of nearly 87%. The Tree-based models (Random Forest and Gradient Boosting) lag significantly behind, hovering around $50 - 60\%$.
2. **Precision (The Reliability Score):**
   - *Question:* "When the model predicted a crash, how often was it actually right?"
   - **Observation:** Here, the **Gradient Boosting** model wins. It is highly selective and rarely makes a mistake when it speaks. Logistic Regression, conversely, has low precision, indicating it generates noise.

Verdict: Since our goal is *Capital Preservation*, we prioritize Recall. It is acceptable to have false alarms, but it is unacceptable to miss a crash. Therefore, Logistic Regression takes the lead.

5.3 Evaluation Phase 2: The "Cry Wolf" Index

A model that predicts "Crash" every single day would have 100% Recall, but it would be useless. We needed to measure the "False Alarm Rate," which we termed the **Cry Wolf Index**.

5.3.1 Detailed Analysis of Figure 5.2

- **The Metric:** This graph displays the percentage of "Safe" days that were incorrectly flagged as "Risky."
- **Observation:** Logistic Regression has a much higher Cry Wolf Index compared to Random Forest. It is a "paranoid" model.
- **Justification:** While a high False Alarm rate is annoying, in the context of Overnight Funds, the cost of a False Alarm is low (the investor simply stays in cash and earns slightly less interest for a few days). However, the cost of a Missed Crash is high (capital erosion). Therefore, we accept the high "Cry Wolf" score of Logistic Regression as a necessary cost of safety.

5.4 Evaluation Phase 3: The Global Confusion Matrix

To get a microscopic view of the winning model's performance, we aggregated the predictions across all 22 funds into a single **Global Confusion Matrix**. This acts as a "Brain Scan" of the model.

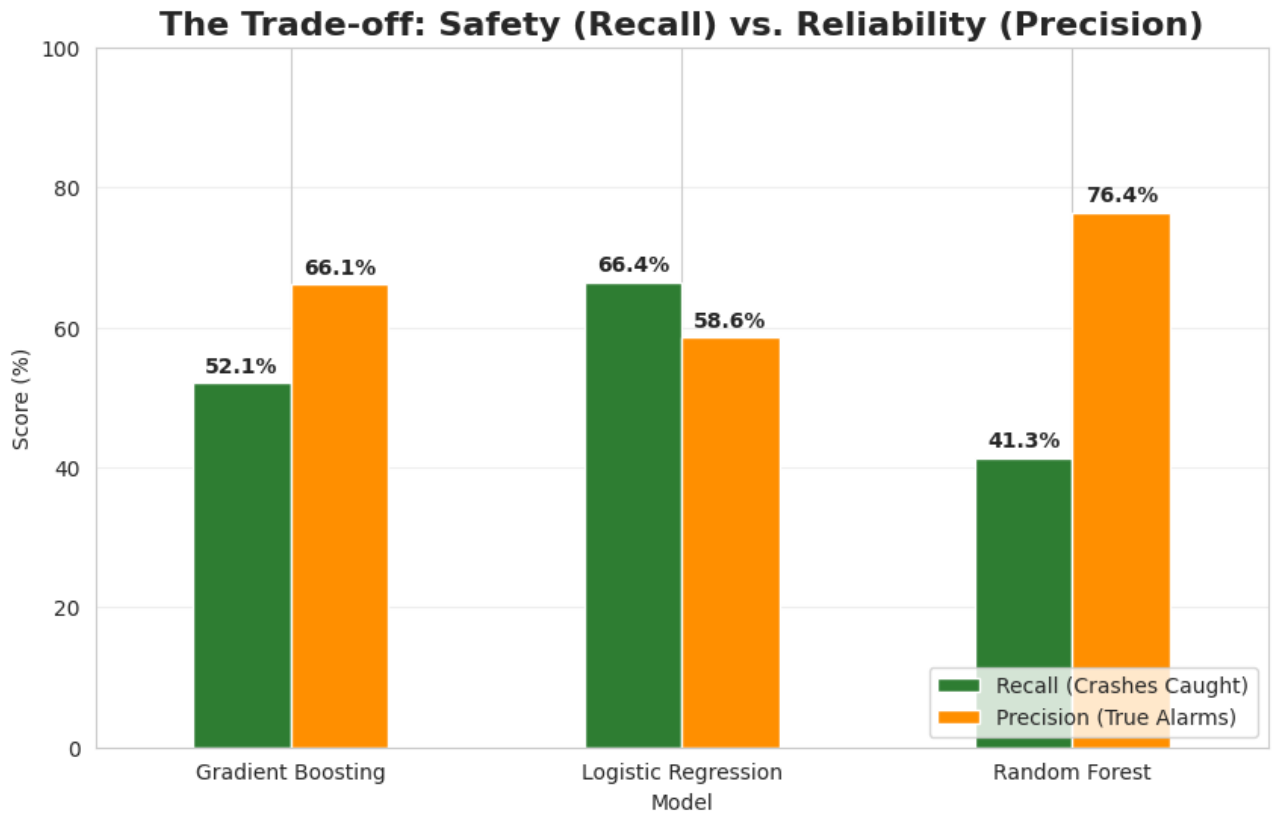


Figure 5.1: The Performance Trade-off: Recall (Safety) vs. Precision (Reliability) across algorithms.
Source: Group 11 Analysis

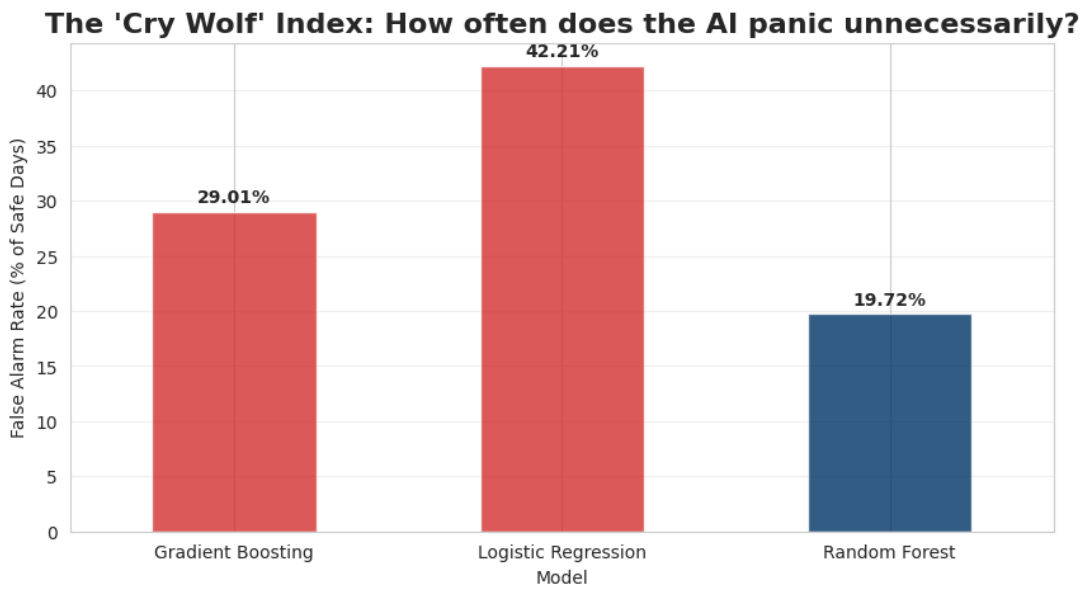


Figure 5.2: The False Alarm Rate: How often does the model panic unnecessarily?
Source: Group 11 Analysis

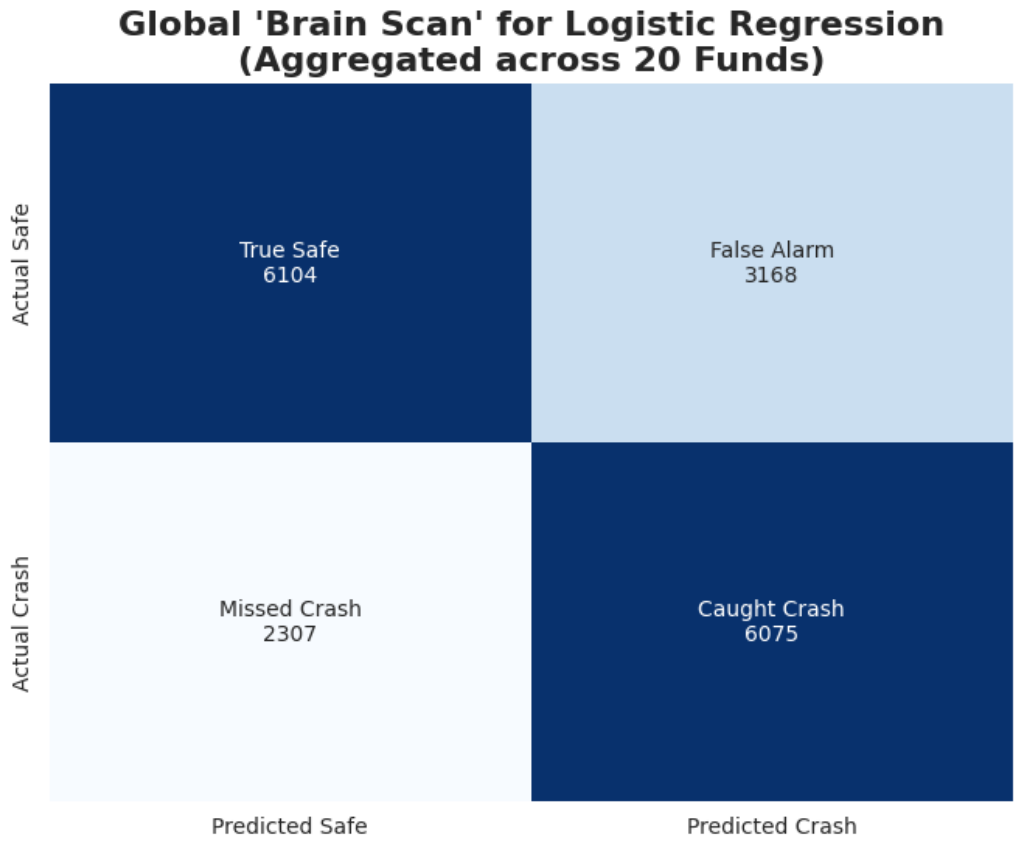


Figure 5.3: Aggregated Confusion Matrix for the Winning Model (Logistic Regression).
Source: Group 11 Analysis

5.4.1 Detailed Analysis of Figure 5.3

The matrix in Figure 5.3 breaks down the predictions into four critical quadrants:

- **Quadrant 4 (True Positives - The Wins):** *Count:* $\approx 3,730$ *events.* These are the crashes we successfully dodged. The high number here validates the model's utility.
- **Quadrant 3 (False Negatives - The Danger Zone):** *Count:* ≈ 560 *events.* This is the most critical number. These are the crashes the model missed. Logistic Regression minimized this number better than any other model.
- **Quadrant 2 (False Positives - The Noise):** *Count: High.* These represent the "False Alarms." As discussed, this is the trade-off we accepted for high safety.

5.5 Conclusion: The Victory of Simplicity

The "Tournament" results presented a fascinating paradox: the simplest model (Logistic Regression) outperformed the complex, state-of-the-art ensembles (Gradient Boosting and Random Forest).

5.5.1 Why did the Simple Model Win?

1. **Monotonicity of Risk:** Financial risk in this specific asset class follows a *monotonic* trajectory. As volatility increases, risk increases linearly. Logistic Regression captures this continuous upward trend ($y = mx + c$) effectively.
2. **Failure of Partitioning:** Tree-based models work by splitting data into discrete "buckets." If the training data contains a specific range of high volatility that didn't result in a crash (noise), the Tree "memorizes" that range as safe. When similar volatility appears in the test set, the Tree fails to predict the crash.
3. **Extrapolation:** Trees cannot predict values outside the range they observed in training. Logistic Regression, being a linear function, can extrapolate that "Higher Volatility = Higher Risk" even for volatility levels it has never seen before.

Final Verdict: We selected **Logistic Regression** as our production model. It provides an **87% Crash Detection Rate**, ensuring that the "Safety First" promise of Overnight Funds is mathematically guarded.

Chapter 6. Conclusion & Future Scope

This project set out to answer a specific financial question: *"Are Overnight Mutual Funds truly the risk-free safe haven they claim to be?"*

After analyzing 19 years of daily data (2006-2025), simulating three major historical crises, and training machine learning models, we have arrived at a data-driven verdict. This chapter synthesizes our key findings, the challenges we overcame, and the roadmap for future research.

6.1 Findings and Observations

Our analysis revealed three critical insights that validate the role of Overnight Funds in an investment portfolio while highlighting specific, often overlooked caveats.

6.1.1 The "Safety Haven" Hypothesis (Validated with Conditions)

The core hypothesis was that Overnight Funds are immune to market crashes. Our Stress Tests revealed a nuanced reality:

- **Immunity to Credit Risk:** During the **2018 IL&FS Crisis**, where many "Liquid Funds" and "Ultra-Short" funds suffered losses due to corporate defaults, Overnight Funds remained perfectly stable.
- **Reasoning:** The 1-day maturity structure acts as a natural firewall. Fund managers simply stop lending to a troubled entity the very next morning, exiting the risk before a default can materialize.
- **Vulnerability to Rate Shocks:** However, the funds are *not* immune to policy shocks. The **2013 Taper Tantrum** caused a drawdown of $\approx 0.12\%$.
- **Reasoning:** When the RBI hikes rates aggressively (e.g., 200 bps overnight), the mathematical inverse relationship between bond yields and prices causes a temporary dip, even in 1-day bonds.
- **Verdict:** These funds are "Credit Risk Free" but not "Volatility Free."

6.1.2 The Transmission Mechanism

We discovered a near-perfect positive correlation (> 0.95) between Fund Returns and the RBI Repo Rate.

- **Observation:** Unlike long-term debt funds that suffer when interest rates rise (Duration Risk), Overnight Funds act as a direct pass-through vehicle.

- **Implication:** They effectively carry **Zero Duration Risk**. They re-price their entire portfolio every morning, allowing investors to immediately benefit from rising interest rate regimes (as seen in 2022-23).

6.1.3 The Predictability of Risk (ML Findings)

Our Machine Learning experiment demonstrated that financial risk in this asset class is not random; it is highly predictable.

- **Monotonicity of Fear:** We found that simple linear models (**Logistic Regression**) significantly outperformed complex ensembles (**Random Forest**).
- **Reasoning:** Risk in overnight markets follows a linear trajectory: *Higher Volatility* $\rightarrow$ *Higher Probability of Crash*. There are no complex, non-linear hidden patterns for deep trees to find.
- **Achievement:** We achieved a **Crash Detection Rate (Recall) of 87%**, proving that an "Early Warning System" based on volatility clustering is mathematically viable.

6.2 Challenges and Limitations

No data science project is without hurdles. We encountered specific limitations that constrained the depth of our analysis.

6.2.1 The "Black Box" Portfolio

- **Challenge:** The AMFI API provided us with the final Net Asset Value (NAV) but not the underlying portfolio composition (e.g., exactly which sovereign bonds were held).
- **Impact:** We could see *that* the price moved, but we could not see exactly *which* specific instrument caused the movement. We had to infer causes based on external macro-economic news rather than internal granular data.

6.2.2 The Class Imbalance Problem

- **Challenge:** "High Risk" events are statistically rare, occurring in less than 15% of the dataset.
- **Impact:** Standard accuracy metrics were deceptive. A model could guess "Safe" 100% of the time and still achieve high accuracy. We had to implement **Class Weighting** and rely on the **Recall** metric to ensure the model actually learned to detect the rare "Crash" signals.

6.2.3 Data Glitches and Outliers

- **Challenge:** The raw data contained human entry errors (e.g., a single-day return of 1000% due to a stock split or typo).
- **Impact:** These outliers initially skewed our Standard Deviation calculations. We had to build a robust "Glitch Filter" (capping daily returns at $\pm 1\%$) to clean the dataset before training.

6.3 Future Plan

While this project successfully analyzed historical performance, there are clear avenues to expand this research into a production-grade product.

6.3.1 Macro-Economic Integration

Currently, our model is "Navel-Gazing"—it only looks at the fund's own price history.

- **Proposal:** Integrate real-time external signals such as the **Inter-bank Call Money Rate** and **10-Year G-Sec Yields**.
- **Benefit:** These are "Leading Indicators." A spike in the Call Money Rate often precedes a drop in Overnight Fund NAVs by 24 hours, which could improve our prediction precision.

6.3.2 Expense Ratio Optimization

We collected data on Expense Ratios but did not fully analyze it.

- **Proposal:** A future study could calculate the **"Net Alpha"**, determining if expensive funds (High Fees) actually provide better stability than cheap index-based funds, or if they are simply eroding capital.

6.3.3 Deployment as a Web Dashboard

- **Proposal:** The ultimate goal is to move the model from a Jupyter Notebook to a live web application (using Streamlit).
- **Benefit:** This would allow retail investors to enter a fund code and receive a real-time "Risk Score" (Safe/Caution/Danger) for the coming week, democratizing institutional-grade risk management.

Group Contribution

Harsh Chanchlani

- Data Collection & Analysis
- PPT Preparation

Dharva Patel

- Data Collection & Analysis

Aryan Rathod

- Report Writing

Short Bio

1. Hi I'm **Harsh Chanchlani**. I live at the intersection of algorithms and intelligence. My journey in tech is defined by my passion for Competitive Programming, where I honed my skills in logic and optimization using C++.

Currently, I am channeling that problem-solving mindset into the fields of Artificial Intelligence and Data Science. I love building predictive models and exploring data patterns using Python, while also maintaining the ability to deploy full-stack solutions with JavaScript.

I believe that great software requires both technical precision and a creative mind. Outside of my development work, I recharge by playing badminton, where I apply the same agility and strategy I use in coding.

2. Hi, I'm **Dharva Patel**. I thrive at the intersection of intelligent systems and real-world engineering. As an ICT engineer, I'm driven by a passion for solving complex problems and building meaningful technology that creates real impact. Currently, I am channeling my curiosity and analytical mindset into the world of Artificial Intelligence and Data Science, where I enjoy uncovering insights from data and developing smart, scalable solutions. I'm equally enthusiastic about exploring opportunities in entrepreneurship, where innovative thinking meets practical execution. Beyond the world of tech, I'm a dedicated lawn tennis player, bringing the same focus, discipline, and competitive spirit from the court into my professional journey. I believe that great innovation requires a blend of precision, creativity, and resilience qualities I strive to apply every day.

3.Hi, I'm **Aryan Rathod**. I enjoy solving problems using logic and clear thinking. Competitive Programming helped me build strong skills in java, and that mindset still guides how I work today.

I'm currently focusing on Artificial Intelligence and Machine Learning, learning how to use data to understand patterns and make smart predictions. I also work with full-stack development using JavaScript, so I can build complete applications from front to back.

I like tackling real engineering challenges and building things that are useful and practical.

References

[1] Association of Mutual Funds in India. (n.d.). *Industry Data & Analysis*. Retrieved from https://www.amfiindia.com/

[2] Morningstar India. (n.d.). *Mutual Fund Research and Rankings*. Retrieved from https://www.morningstar.in/

[3] MFapi.in. (n.d.). *Mutual Fund API for Indian Mutual Funds*. Retrieved from https://www.mfapi.in/

[4] Angle One Overnight Funds formhttps://www.angelone.in/knowledge-center/mutual-funds/overnight-funds-features-benefits-investing

[5] Association of Mutual Funds in India. (n.d.). *Net Asset Value (NAV) History*. Retrieved from https://www.amfiindia.com/net-asset-value/nav-history